

TD02 : Fonctions et Jeux

Adrien ESCOURROU

November 21, 2025

INTRODUCTION

Dans ce TD02, nous étudierons le concept de fonction en tant que sous-programme réutilisable capable de produire des résultats. Nous verrons comment définir des fonctions en Python, les conventions de retour de valeurs et, plus particulièrement, comment une fonction peut renvoyer plusieurs résultats via des valeurs tuple, listes ou dictionnaires. L'objectif est de comprendre quand et pourquoi privilégier un seul retour vs plusieurs retours, et d'appliquer ces notions dans des exercices pratiques.

Rappels essentiels

Une fonction a pour syntaxe :

```
1  def nomFonction (parametres) :  
2      """ Documentation de la fonction """  
3      bloc instructions  
4      return resultats
```

*Tous vos résultats devront être publiés sur votre répertoire GIT partagé avec votre professeur en TP01. Ici vous nommerez un répertoire **TD02** et annoterez chaque code avec l'exercice associé. Par exemple, pour l'exercice 3 vous publierez **Exo3.py** dans **TD02**.*

Exercice 1

Écrire une fonction **majorite** qui prend en paramètre le prénom et l'âge d'une personne. La procédure affichera ensuite un message disant '*Bonjour prénom*', puis indiquera s'il est mineur (-18) ou s'il est majeur.

Exercice 2

Écrire une fonction **volumeSphere** qui calcule le volume d'une sphère de rayon r fourni en argument. Tester la fonction volumeSphere par un appel dans le programme principal.

NB : Volume d'une sphère = $\frac{4}{3}\pi r^3$

Exercice 3

Écrire une fonction **airePerimRect** qui calcule l'aire et le périmètre d'un rectangle.

*NB : Aire = Longueur $\times$ Longueur
Périmètre = (Longueur + largeur) $\times$ 2*

INFO : En Python, une fonction peut retourner plusieurs valeurs !

Exercice 4

Écrire une fonction **supprimElementsNegatifs** prenant une liste d'entiers et supprimant les entiers strictement négatifs de cette liste. Tester cette fonction.

INFO : Une fonction peut agir sur ses paramètres (peut retourner un résultat différent de ses paramètres).

*NB : Aire = Longueur $\times$ Longueur
Périmètre = (Longueur + largeur) $\times$ 2*

INFO : En Python, une fonction peut retourner plusieurs valeurs !

Jeu « Nombre mystère »

On veut programmer un petit jeu en console. L'ordinateur choisit au hasard un nombre entier dans un intervalle (par exemple entre 1 et 100). Le joueur doit deviner ce nombre et à chaque proposition, l'ordinateur indique :

- « Trop petit »
- « Trop grand »
- ou « Bravo ! »

Le jeu se termine lorsque le joueur trouve le nombre. On affiche alors le nombre d'essais. Vous devez structurer votre programme en plusieurs fonctions.

Objectifs :

- Savoir définir et utiliser des fonctions
- Manipuler les entrées clavier (input)
- Utiliser les boucles (while) et les conditions (if/elif/else)
- Utiliser le module random

Contraintes :

Le programme doit utiliser au minimum ces fonctions :

1. **demander_limites()**

- Ne prend aucun paramètre
- Demande à l'utilisateur une borne minimale et une borne maximale
- Renvoie un tuple (mini, maxi) d'entiers
- On doit vérifier que $\text{mini} < \text{maxi}$ (redemander si ce n'est pas le cas).

2. **tirer_nombre_mystere(mini, maxi)**

- Prend en paramètres les bornes mini et maxi
- Renvoie un entier tiré au hasard dans cet intervalle (inclus).

3. `demander_proposition(mini, maxi)`

- Demande au joueur de proposer un nombre entre mini et maxi
- Vérifie que l'entrée est un entier valide dans l'intervalle
- Tant que ce n'est pas valide, redemande.
- Renvoie l'entier proposé.

4. `analyser_proposition(proposition, secret)`

- Compare proposition et secret
- Affiche « Trop petit », « Trop grand » ou « Bravo ! »
- Renvoie :
 - -1 si la proposition est trop petite
 - 1 si elle est trop grande
 - 0 si elle est correcte

5. `jouer_une_partie()`

- Gère une partie complète :
 - Demande les limites
 - Tire le nombre mystère
 - Boucle jusqu'à ce que le joueur trouve
- À la fin, affiche le nombre d'essais.

6. `demander_rejouer()`

- Demande au joueur s'il veut rejouer (o / n)
- Renvoie True si le joueur veut rejouer, False sinon.

Le programme principal doit :

1. Appeler `jouer_une_partie()` au moins une fois
2. Tant que `demander_rejouer()` renvoie True, relancer une partie.

Bonus (si vous avez le temps) :

- Ajouter une limite max d'essais (par exemple 10).
- Si le joueur n'a pas trouvé au bout de 10 essais, il a perdu.

- Mémoriser le meilleur score (minimum d'essais pour gagner) sur toutes les parties jouées dans la même exécution.

Squelette de code :

```
1 import random
2
3 def demander_limites():
4     """Demande a l'utilisateur mini et maxi, et les
5         renvoie."""
6     # TODO : a completer
7     pass
8
9 def tirer_nombre_mystere(mini, maxi):
10     """Renvoie un entier aleatoire entre mini et maxi
11         inclus."""
12     # TODO : a completer
13     pass
14
15 def demander_proposition(mini, maxi):
16     """Demande une proposition valide entre mini et
17         maxi."""
18     # TODO : a completer
19     pass
20
21 def analyser_proposition(proposition, secret):
22     """
23     Compare proposition et secret.
24     Renvoie :
25         -1 si trop petit
26         1 si trop grand
27         0 si correct
28     """
29     # TODO : a completer
30     pass
31
32 def jouer_une_partie():
33     """Gere une partie complete du jeu."""
34     # TODO : a completer
35     pass
36
37 def demander_rejouer():
38     """Demande au joueur s'il veut rejouer. Renvoie
```

```

    True/False. """
36     # TODO : a completer
37     pass
38
39 # Programme principal
40 if __name__ == "__main__":
41     print("=== Bienvenue dans le jeu du Nombre Mystere ===")
42     # TODO : boucle principale du jeu
43     pass
```

Conclusion

Bravo ! Vous venez d'utiliser vos premières fonctions.